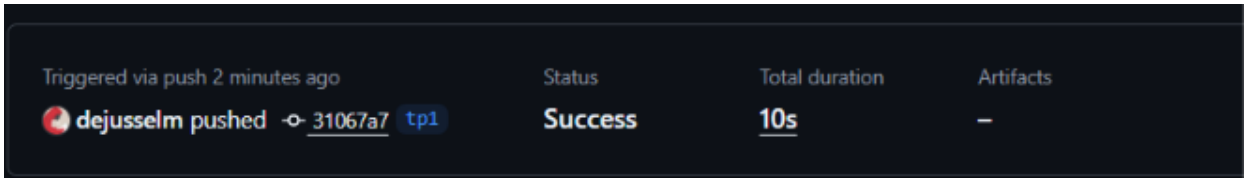
Marine DEJUSSEL

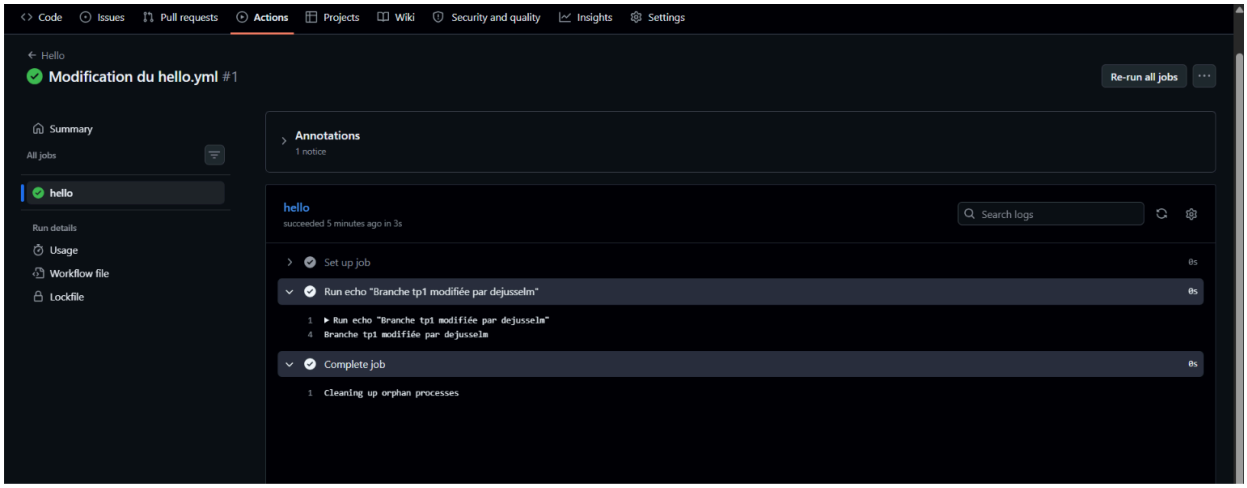
Rendu le : 5 Novembre

Liens vers les repo GitHub : tasks-api | tasks-config

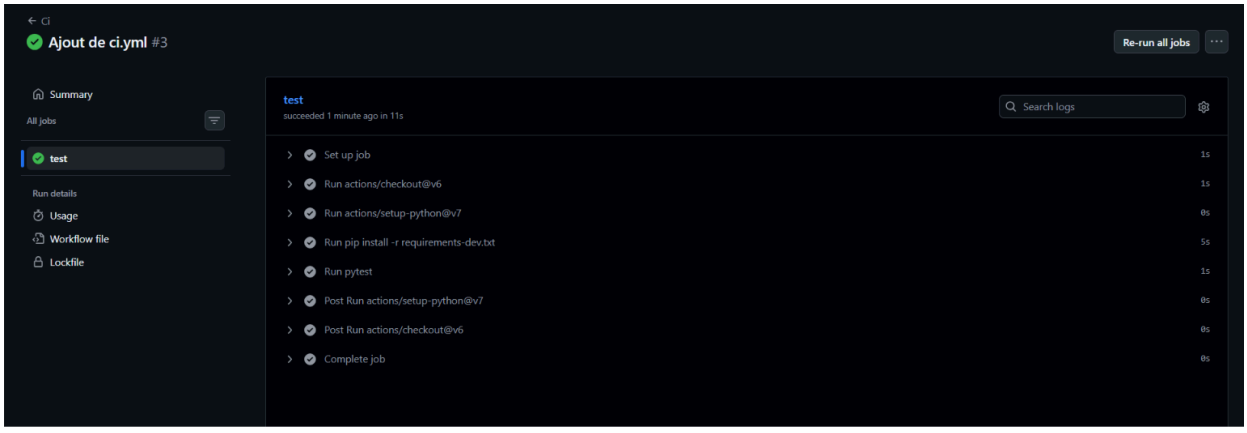
TP1-1 Run Hello



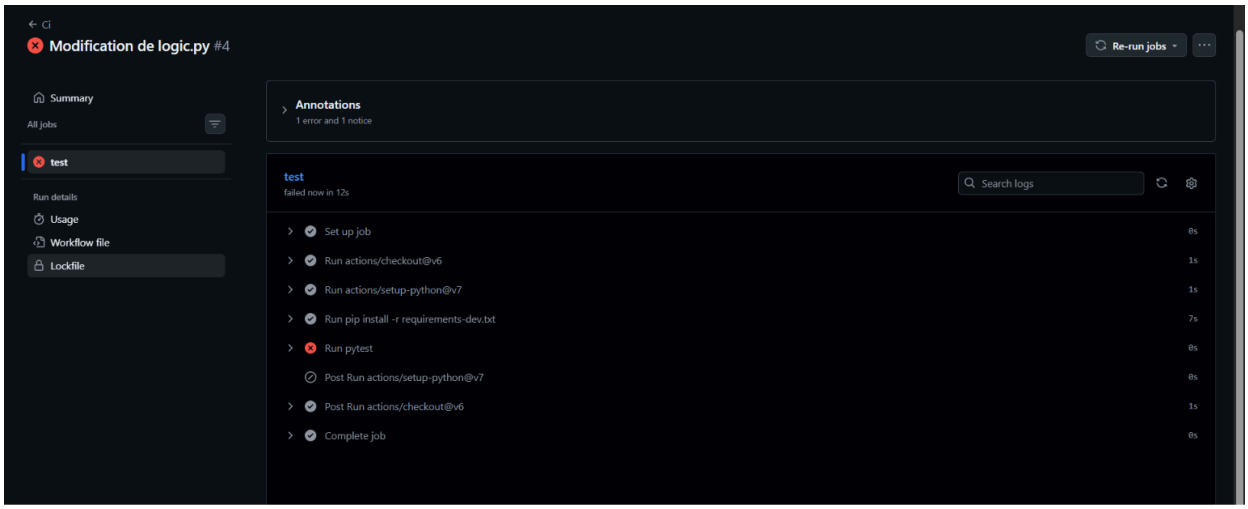
Il a duré 10 secondes au total. On peut voir les log de chaque étapes dans les cases dépliantes de chaque job (ex en cliquant sur “Run echo “Branche tp1 modifiée par dejusselm” dans le job “hello”)

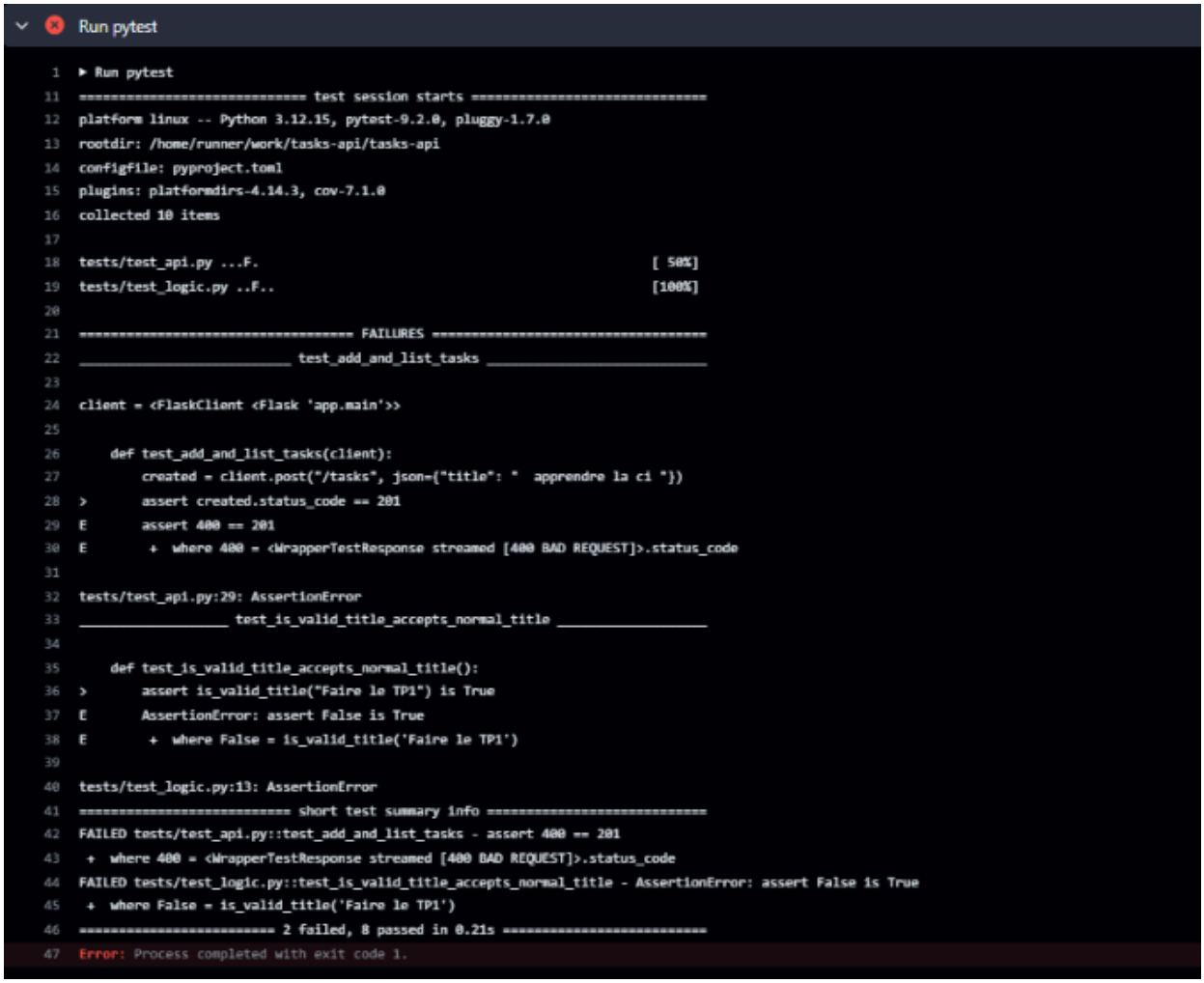


TP1-2 Run ci.yml



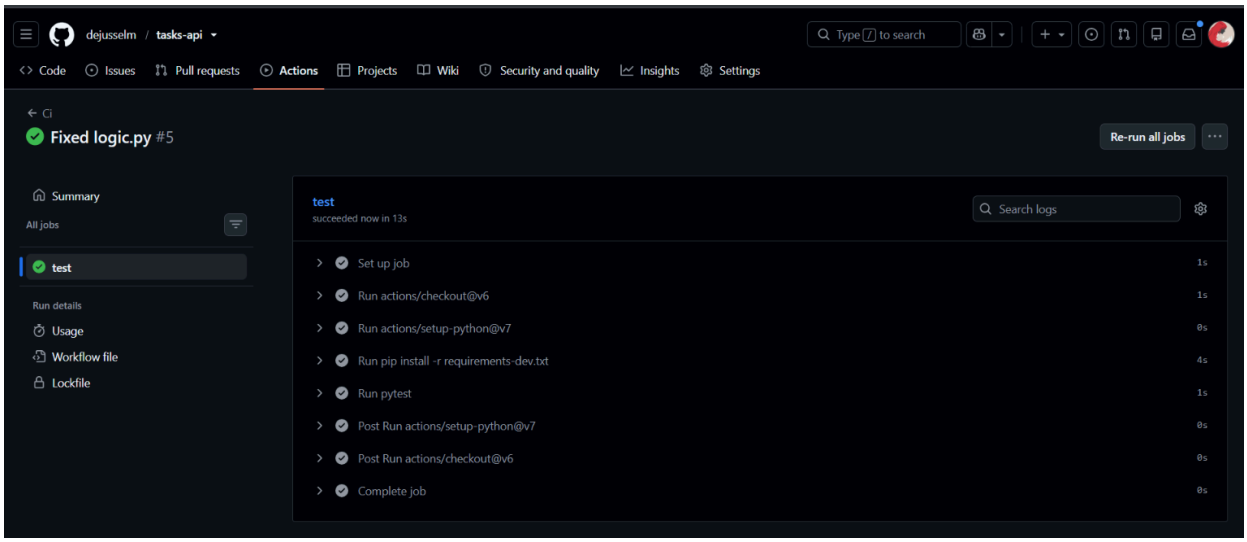
TP-3 Run Rouge



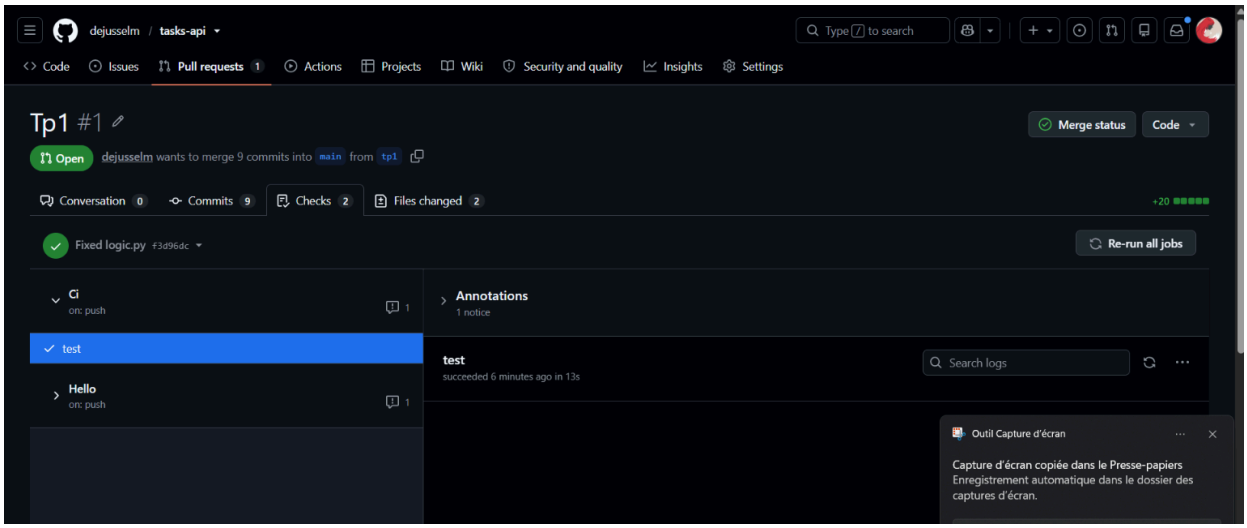


Les tests **test_add_and_list_tasks** et **test_is_valid_title_accepts_normal_title** ont échoué.
test_add_and_list_tasks à la ligne 29 de test_api.py
test_is_valid_title_accepts_normal_title à la ligne 13 de test_logic.py
Les lignes et fichiers concernant le fail du test sont annotés dans les log du job, notamment dans le step “Run pytest”

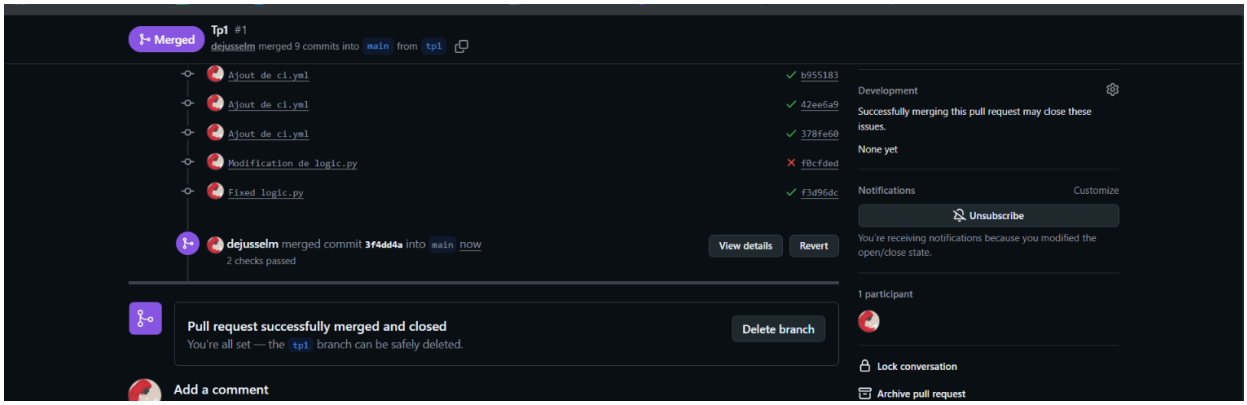
TP1-4 Version corrigée



TP1-5 Pull Request



TP1-6 Pull request fusionnée

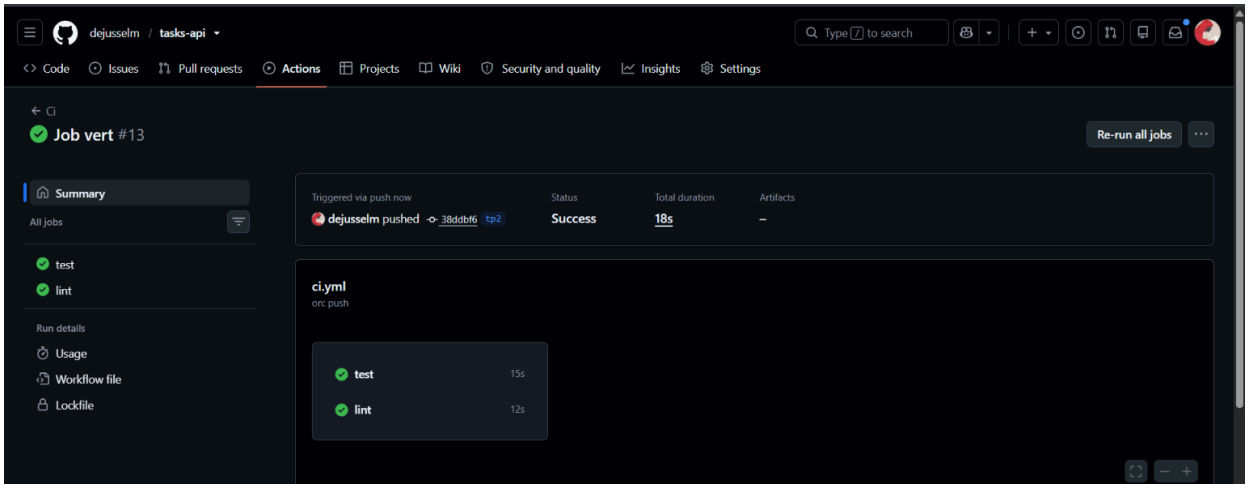


TP1-7 Badge affiché

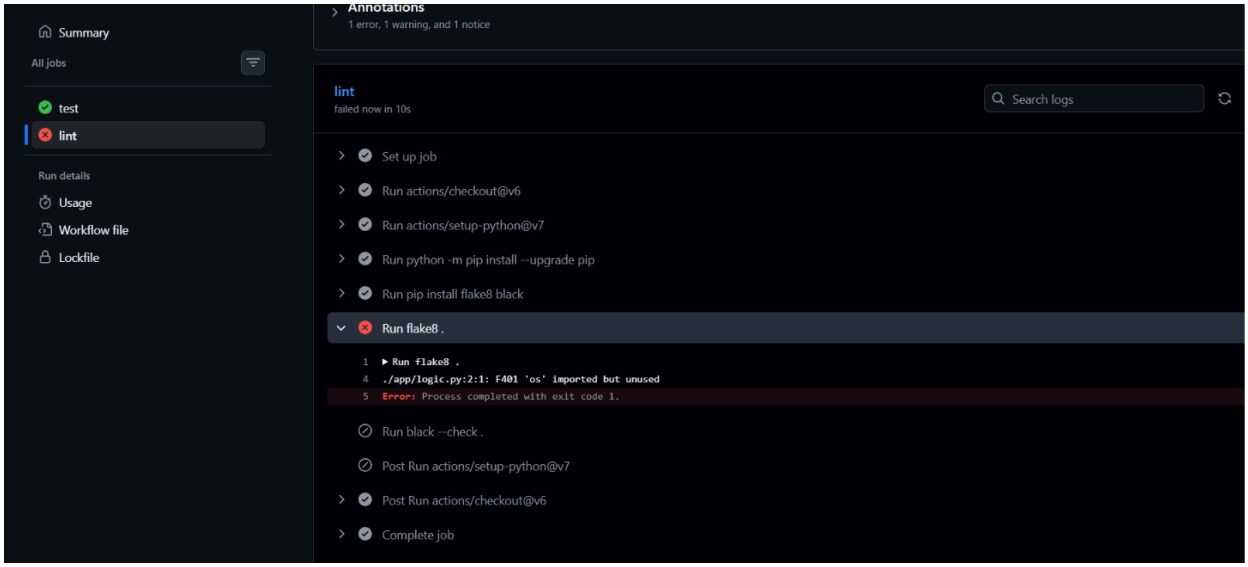


On attend que la CI soit verte pour fusionner car cela indique que tous les tests sont passés, et qu'il n'y a plus de corrections à réaliser. Et si jamais la CI n'est pas verte et qu'il y a trop de problèmes, on peut toujours revenir en arrière à l'état des fichiers sur la branche main.
On peut donc push sur la branche main en sécurité lorsque les CI sont verts.

TP2-1 Job vert

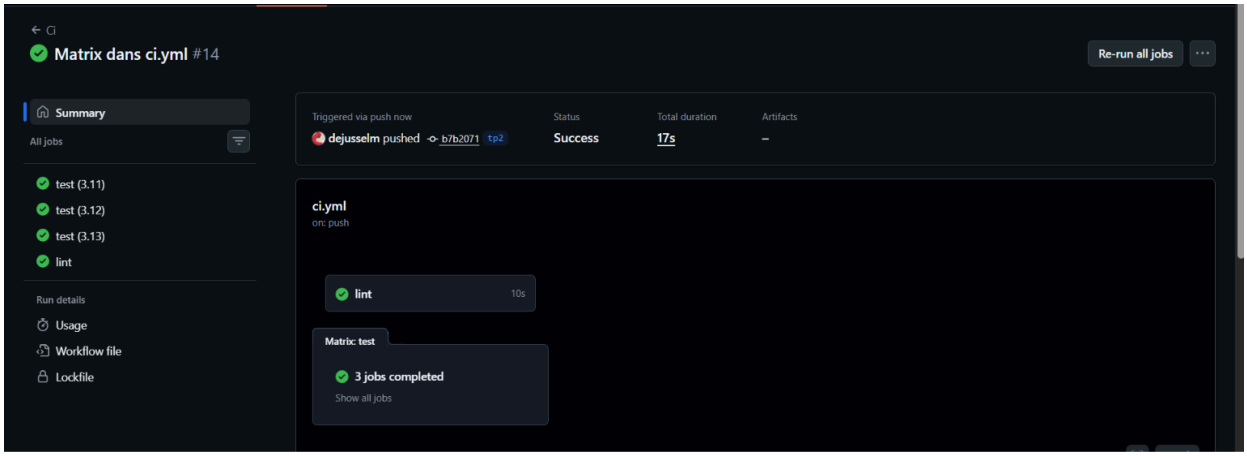


TP2-2 Job Lint rouge

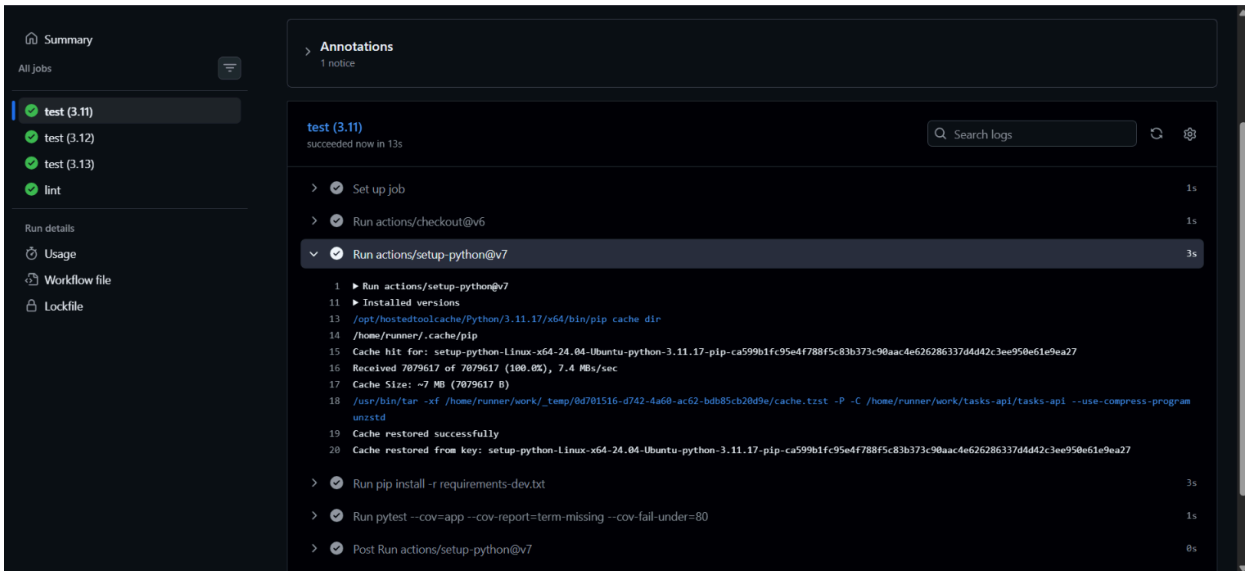


Difference entre linter et formateur : linter évite le code mort, formateur améliore la lisibilité et l'uniformité du code
Séparer lint et test car ils concernent des test différents : un concerne les fonctionnalités, et l'autre le formatage

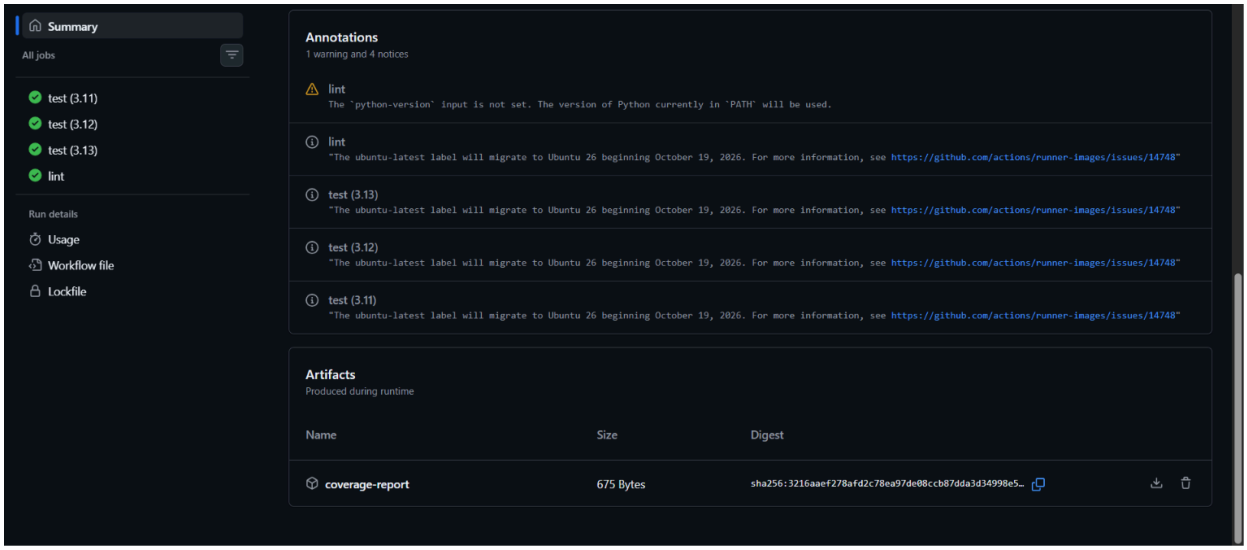
TP2-3 Jobs Matrice



TP2-4 Cache

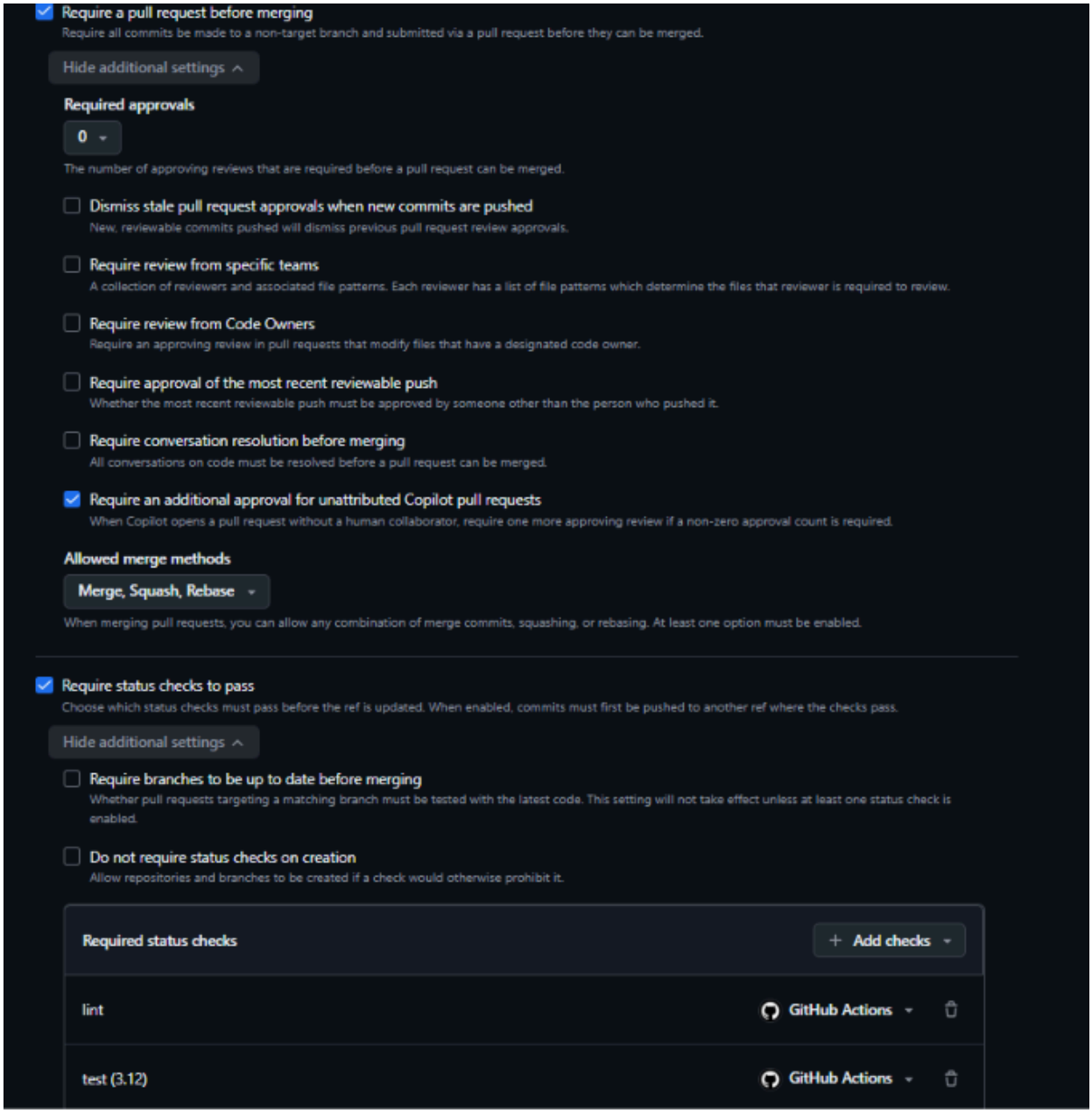


TP2-4 Artéfact

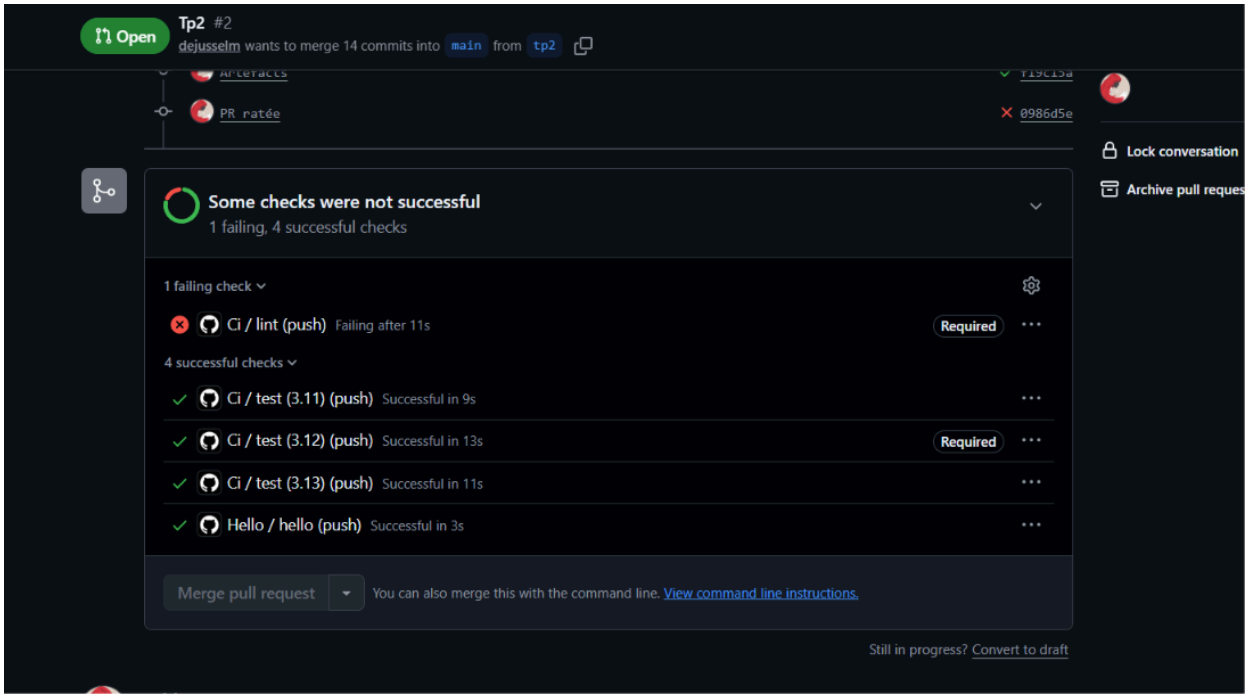


Certaines choses peuvent changer d’une version à une autre de python
Le cache permet d’éviter de perdre du temps à chaque CI, en évitant de réinstaller à chaque fois les dépendances
Un artefact permet de garder le résultat du run (ex : un rapport)

TP2-5 Protection Branche Main

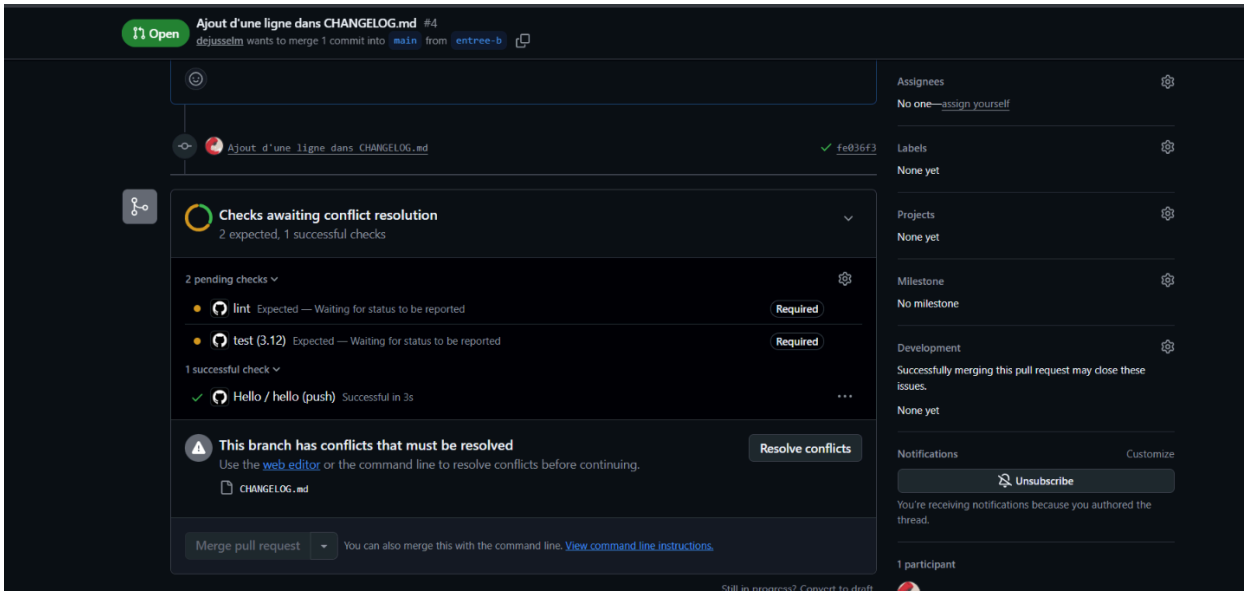


TP2-6 PR ratée

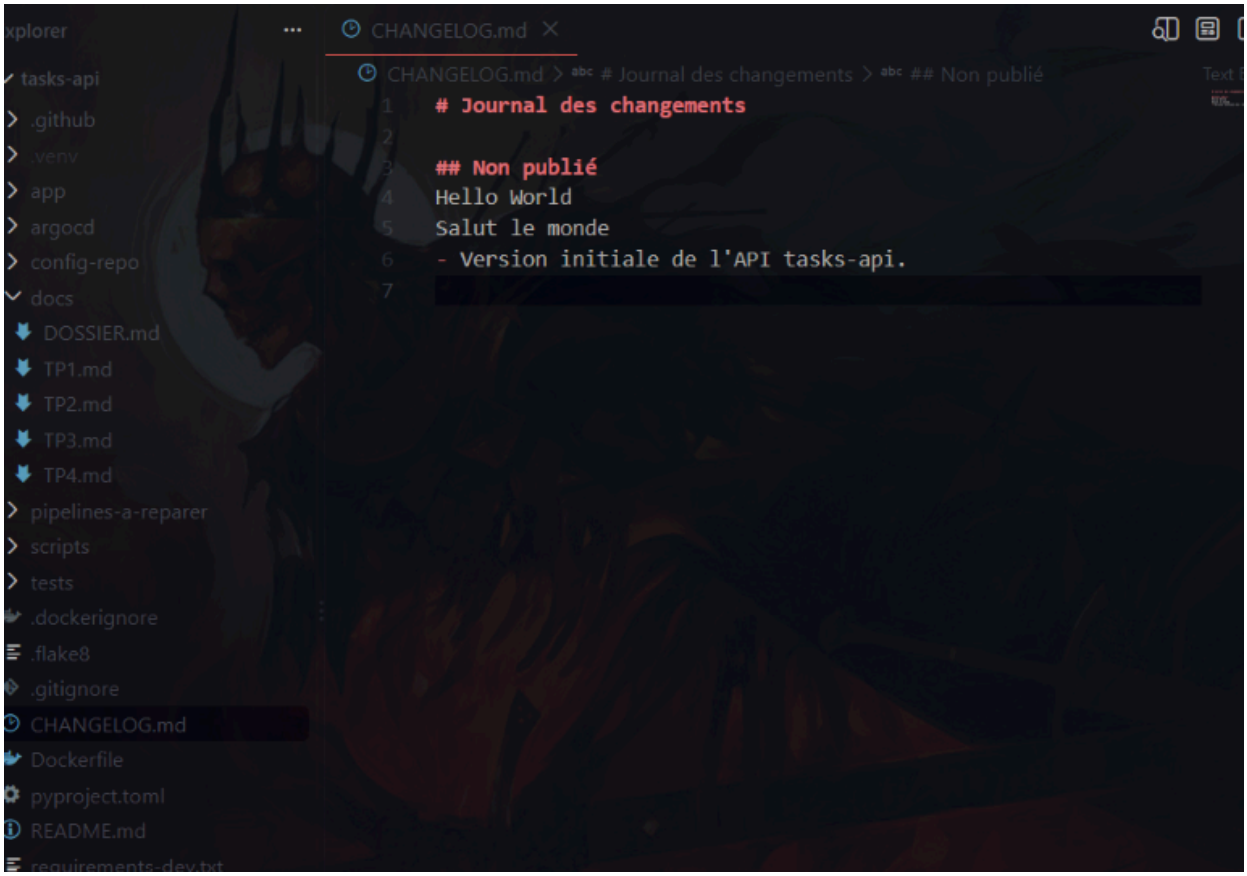


On empêche de push directement sur la branche main pour éviter de perdre une version fonctionnelle, et push des erreurs

TP2-7 PR de B avec conflit

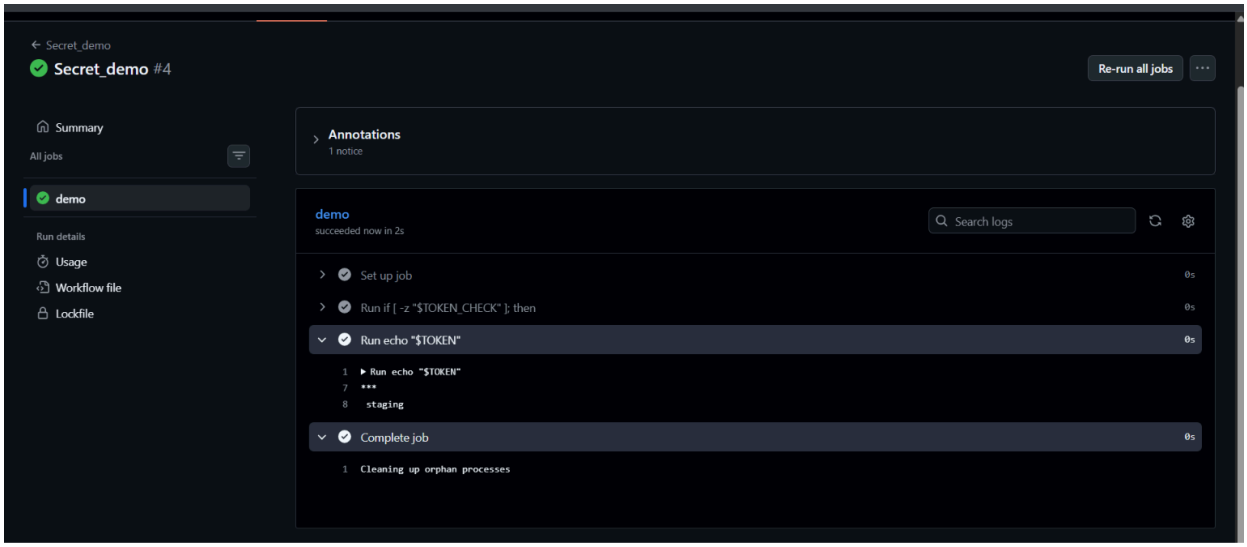


TP2-8 CHANGELOG.md final



Git ne peut pas résoudre le conflit tout seul puisqu'il ne peut pas savoir quelle modification on veut garder, supprimer, etc..
Après la résolution, les CI ont de nouveaux vérifié le code.

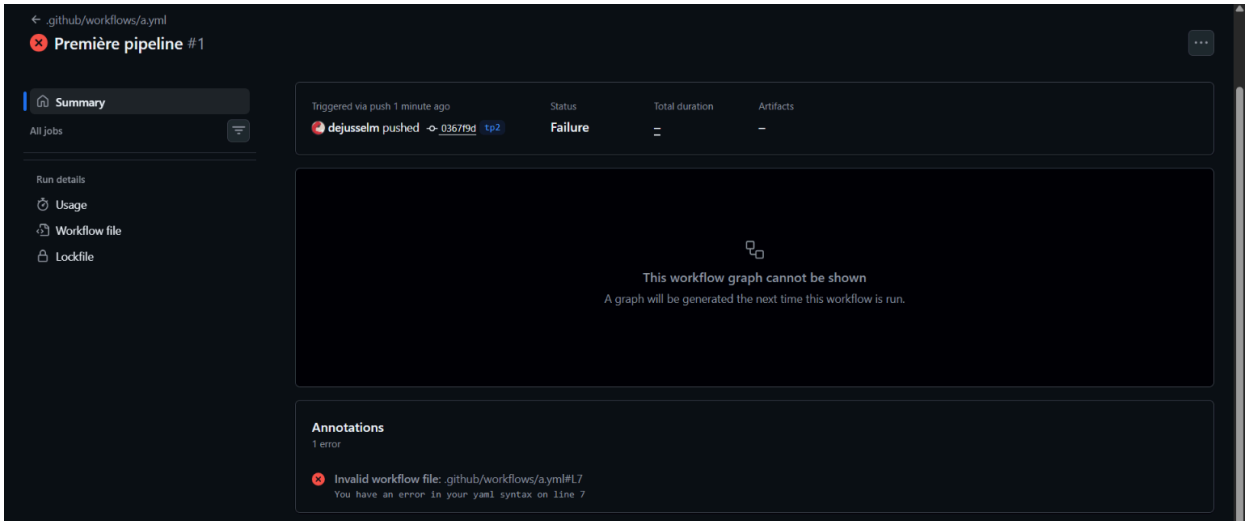
TP2-9 Run Valeur Secret



GitHub masque le secret pour éviter de récupérer facilement une valeur qui peut s'avérer critique
Si on le commit dans le dépot, la valeur est visible par tout ceux qui ont accès au repo
On choisit entre variable et secret en fonction de la confidentialité de la donnée.

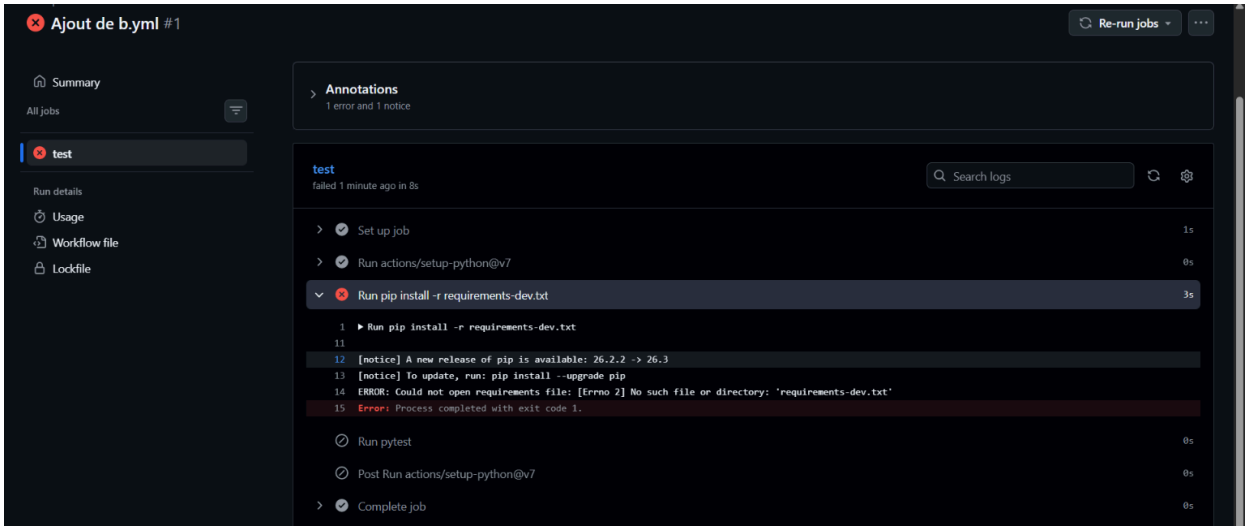
TP2-10 Pipelines à Corriger

TP2-10A



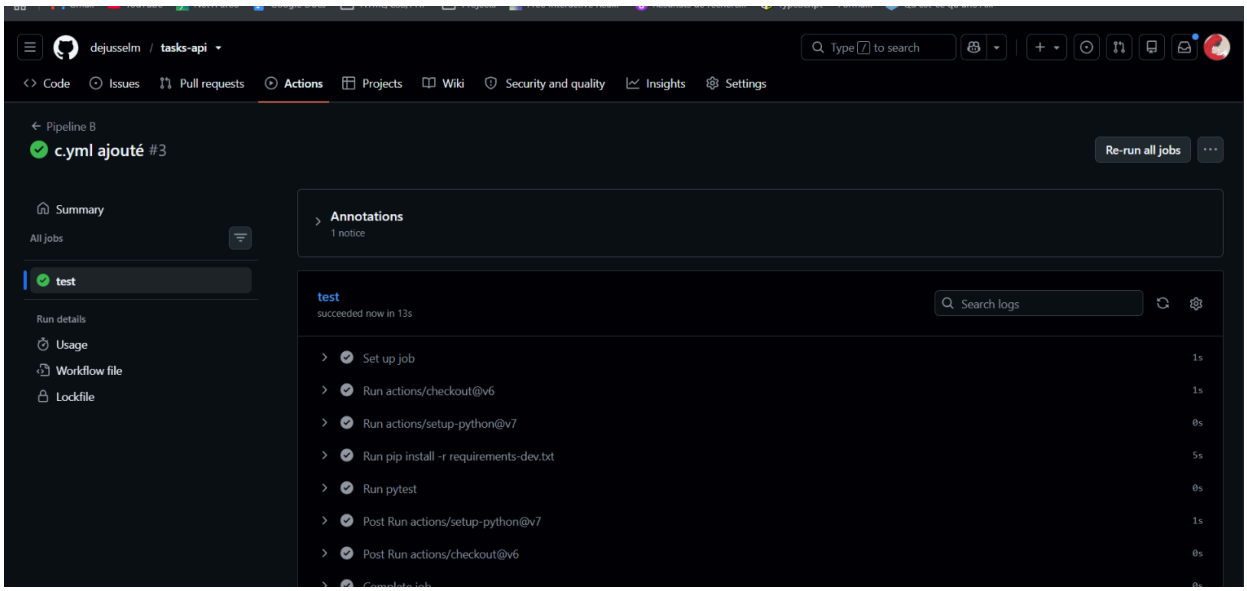
Le pipeline A n'a même pas démarré : erreur de yaml à la ligne 7
Il s'agissait d'un problème d'indentation : `- run: pip install -r requirements-dev.txt` doit être aligné avec les autres run

TP2-10B



Le pipeline a démarré mais s'est interrompu : il ne trouve pas le fichier requirements-dev.txt
Il manquait la ligne - `uses: actions/checkout@v6`
Elle va chercher le code source du repo (qui contient donc requirements-dev.txt)

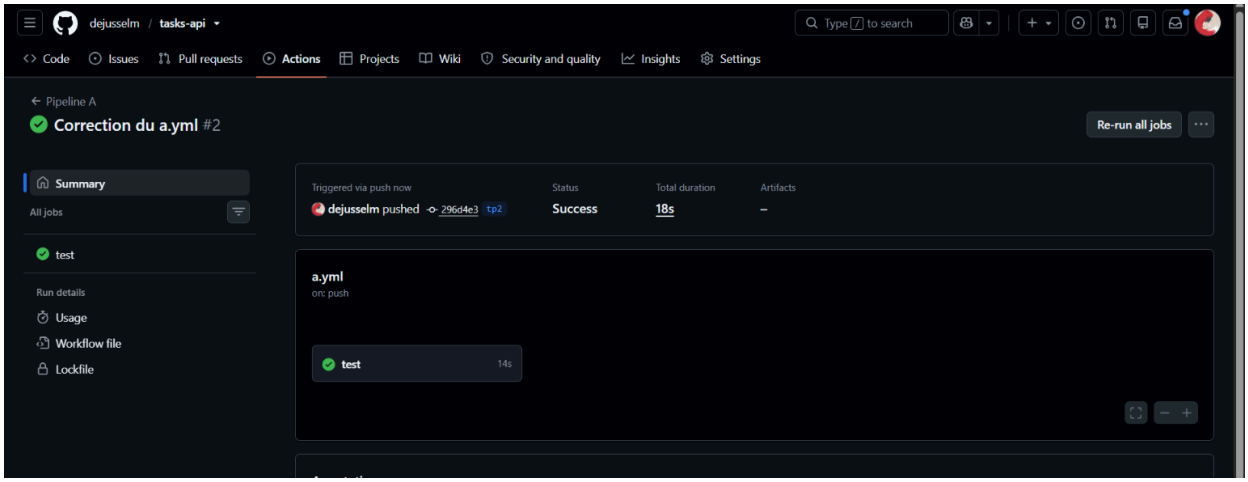
TP2-10C



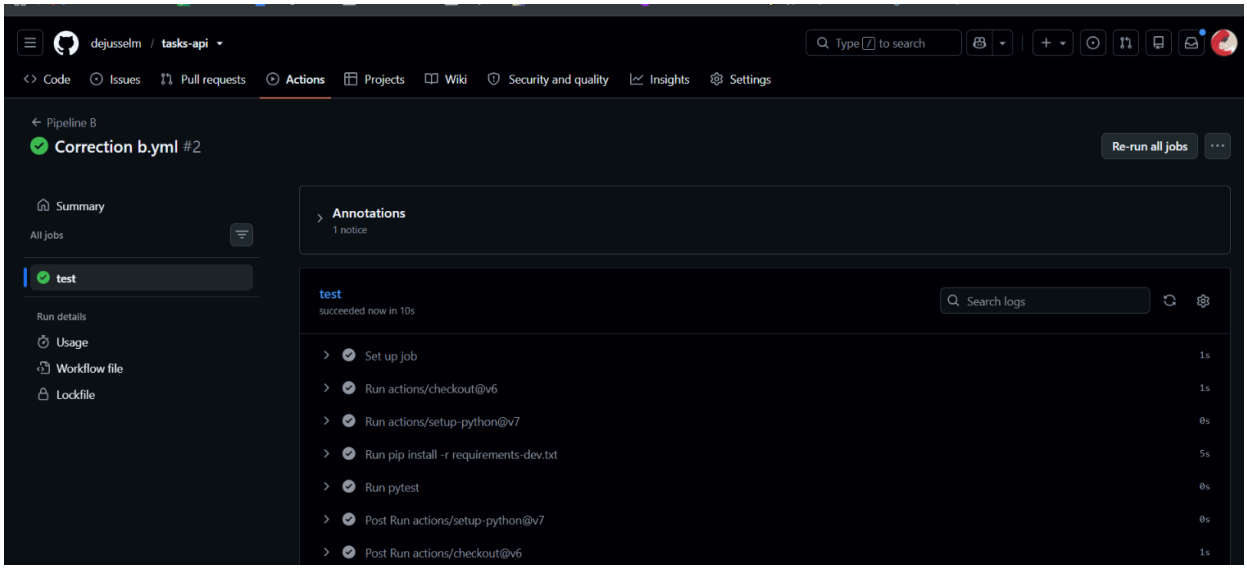
Le pipeline C s'est exécuté sans problème, en revanche un détail est à noter dans le code : dans la matrice, les valeurs ne sont pas entre guillemets. Cela peut poser problème pour les valeurs : 3.10 devient 3.1
La matrice doit être ainsi : ["3.10","3.11","3.12"]

TP2-11 Liste des runs verts

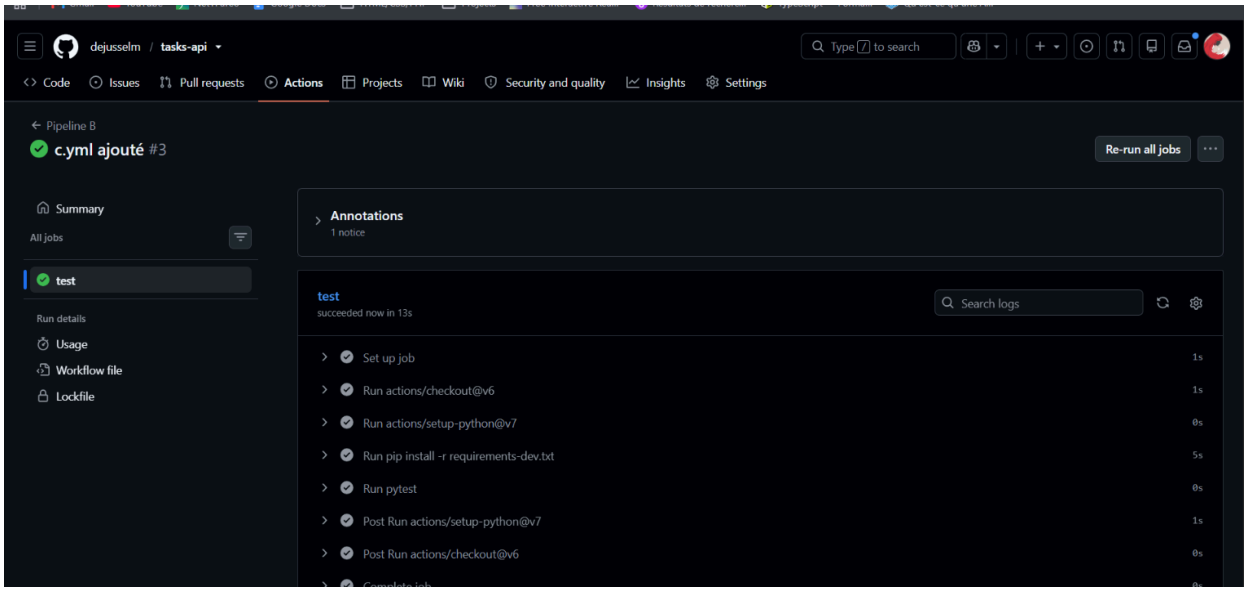
Correction pipeline A



Correction Pipeline B



TP2-10C


dejusselm / tasks-api
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pipeline B
c.yml ajouté #3
Re-run all jobs
Summary
All jobs
test
Run details
Usage
Workflow file
Lockfile
Annotations
1 notice
test
succeeded now in 13s
Search logs
Set up job 1s
Run actions/checkout@v6 1s
Run actions/setup-python@v7 0s
Run pip install -r requirements-dev.txt 5s
Run pytest 0s
Post Run actions/setup-python@v7 1s
Post Run actions/checkout@v6 0s

TP3-1 Build docker réussi

```
(.venv) PS C:\Users\quasc\Documents\Coda\B2\GitOps\tasks-api> docker build -t tasks-api:local .
>> docker run --rm -p 8000:8000 tasks-api:local/health
[+] Building 0.7s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.45kB
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [internal] load .dockerignore
 => => transferring context: 196B
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30
 => [internal] load build context
 => => transferring context: 363B
 => CACHED [2/6] WORKDIR /app
 => CACHED [3/6] COPY requirements.txt .
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/6] COPY app/ app/
 => CACHED [6/6] RUN groupadd --gid 1000 default     && useradd --uid 1000 --gid 1000 -m default
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:2cd6e529bf142892f316d2e05dbc4f25c9c767491beeaedd89a486b6edb4a1f1
 => => exporting config sha256:b8a284788ea284ce1eb07f6d975847b20d4aa456c5c93e4c8e282e9e6d5f0dc3
 => => exporting attestation manifest sha256:4c45bbc2e9d4173104db50e16ce4b9c98956df2269b954400edf26c19fa8ed15
 => => exporting manifest list sha256:98f89bc2d3046d47a9faf9ad2c318656f6862bc6458e6d0f7c2982e0ebcf96ea
 => => naming to docker.io/library/tasks-api:local
 => => unpacking to docker.io/library/tasks-api:local
```

TP3-1 Curl /health

```
PS C:\Users\quasc> curl http://localhost:8000/health

Avertissement de sécurité : risque d'exécution de script
Invoke-WebRequest analyse le contenu de la page web. Il se peut que le code de script de la page web s'exécute lors de
l'analyse de la page.
      ACTION RECOMMANDÉE :
      Utilisez le commutateur -UseBasicParsing pour éviter l'exécution du code de script.

      Voulez-vous continuer ?

[O] Oui  [T] Oui pour tout  [N] Non  [U] Non pour tout  [S] Suspendre  [?] Aide (la valeur par défaut est « N ») : o


StatusCode        : 200
StatusDescription : OK
Content           : {"status":"ok"}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 16
                    Content-Type: application/json
                    Date: Wed, 04 Nov 2026 12:59:13 GMT
                    Server: gunicorn

                    {"status":"ok"}

Forms             : {}
Headers           : {[Connection, close], [Content-Length, 16], [Content-Type, application/json], [Date, Wed, 04 Nov
                    2026 12:59:13 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 16
```

taille image : 201Mb

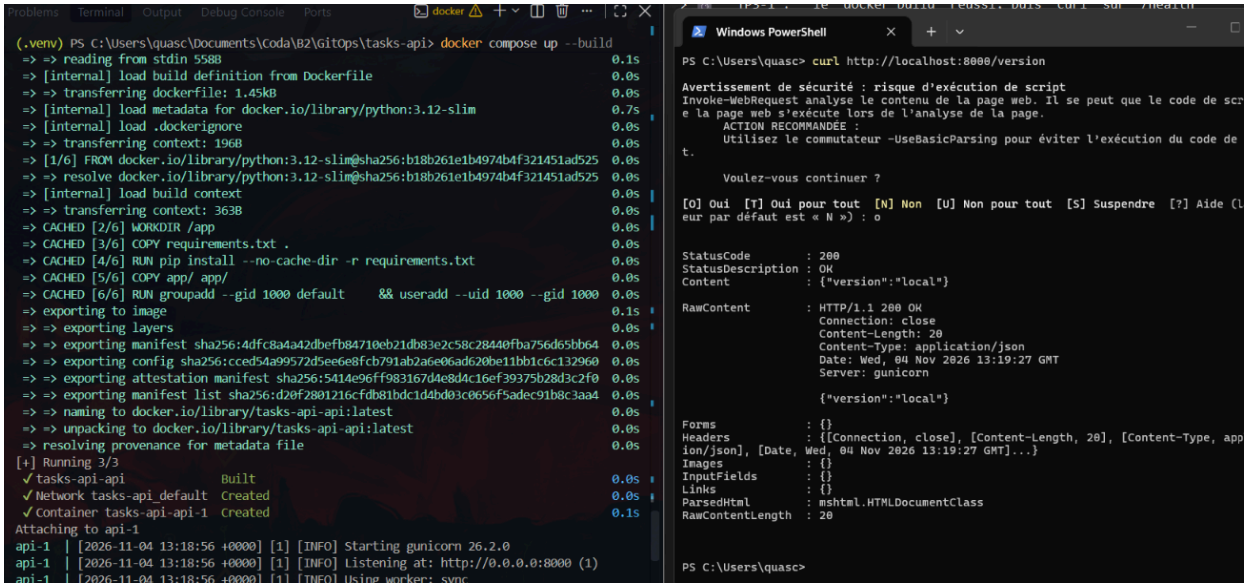
Les étapes CACHED sont WORKDIR, COPY, et RUN pip

Requirements.txt est copié avant le code puisque le code dépend de ces fichiers, cela engendrerait des erreurs. de plus dans un dockerfile, les lignes qui sont le moins souvent liées à des changements sont écrites en premier

tasks-api est une image qu'on a ensuite utilisée pour démarrer un container. L'image est un modèle fixe, tandis que le container est une copie d'image dynamique

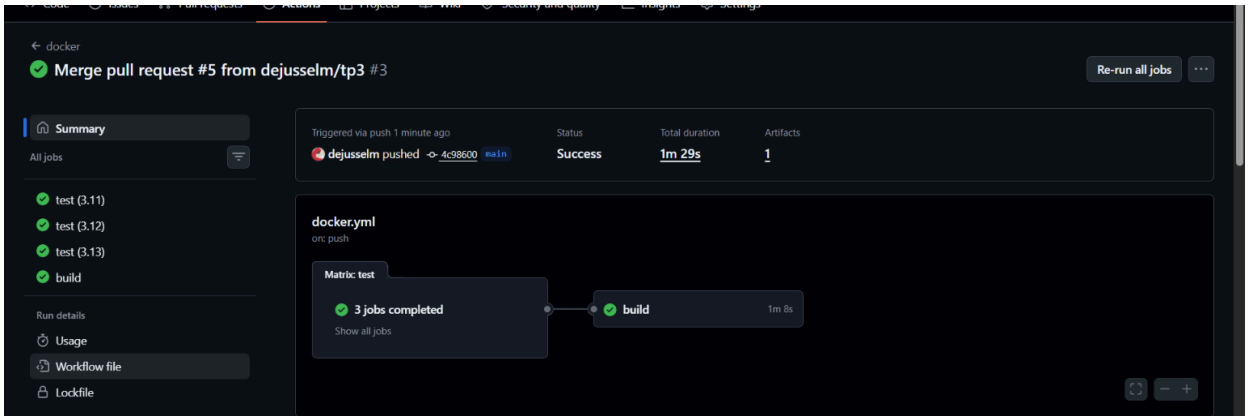
Le .dockerignore est similaire au .gitignore : il ignore lors du build les fichiers désignés, donc évite d'envoyer certains fichiers

TP3-2 Docker compose up et curl /version



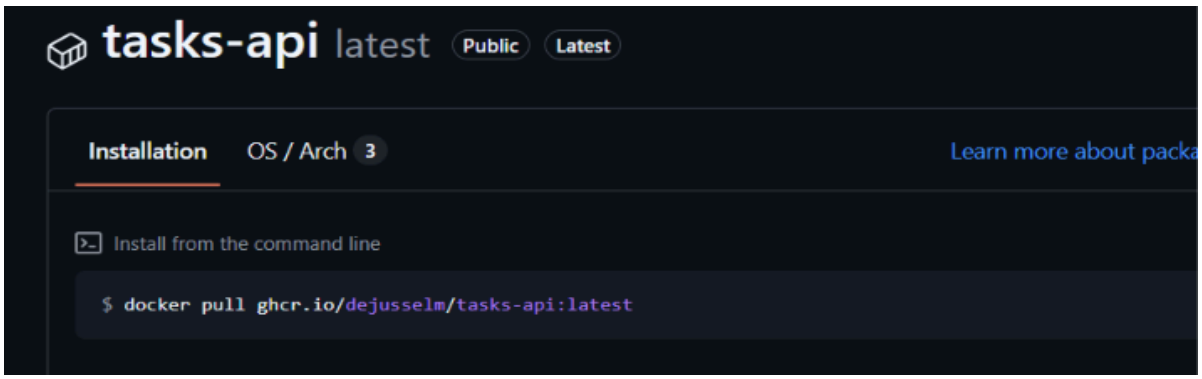
En décrivant le lancement dans un fichier plutôt que dans une longue commande, facilite la compréhension ainsi que la modification de l'ensemble.

TP3-3 Run test puis build vert

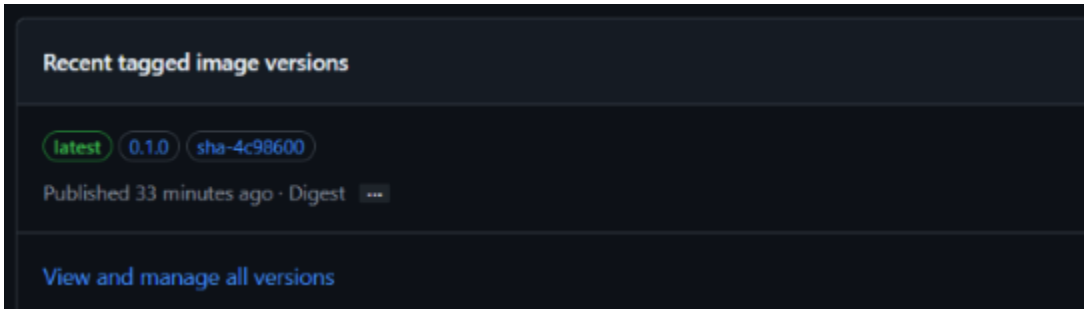


Le job build requiert le passage du job test, afin d'éviter de publier du code défaillant / mort
On évite le tag latest puisqu'il est flou : on ne sait pas exactement à quelle version on se trouve
On publie une image pour deux architectures ces processeurs représentent la majorité : ceux utilisés par les cloud et ceux utilisés par les dev. Lorsque docker télécharge l'image, l'image s'adapte donc à l'architecture

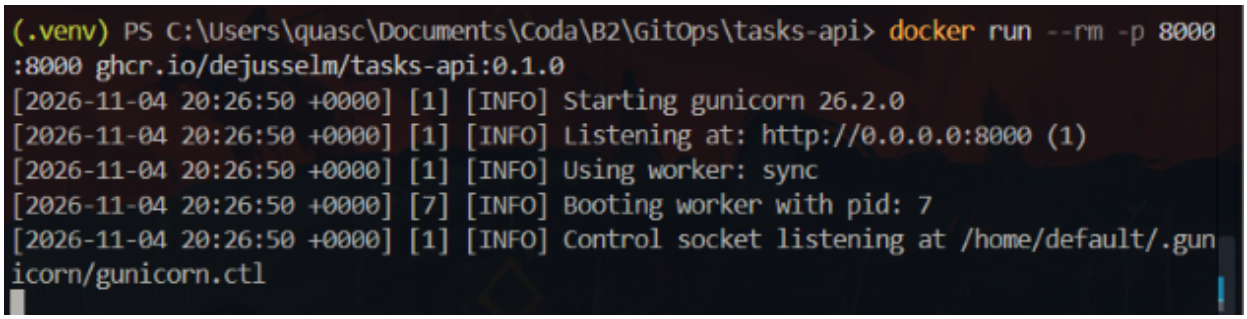
TP3-4 TAG 0.1.0



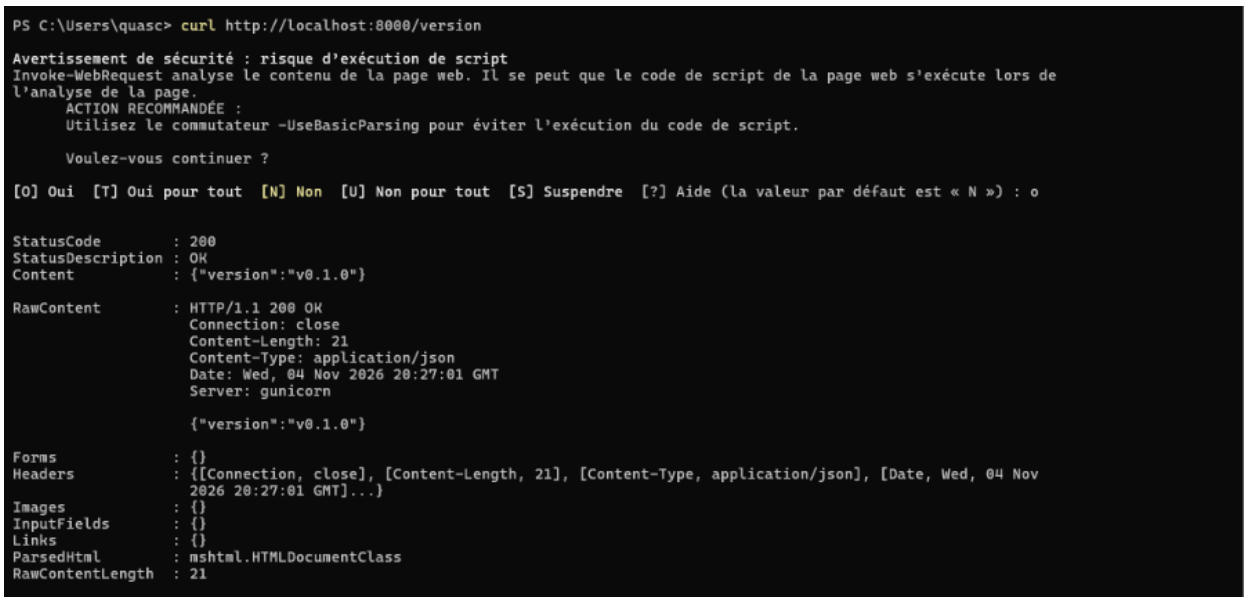
TP3-5 Liste des tags



TP3-6 Docker run de l'image téléchargée

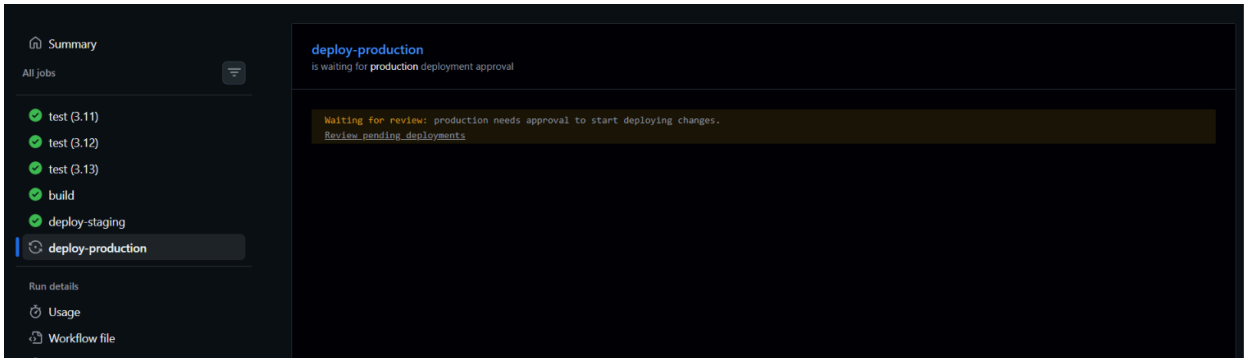


TP3-6 Curl de /version

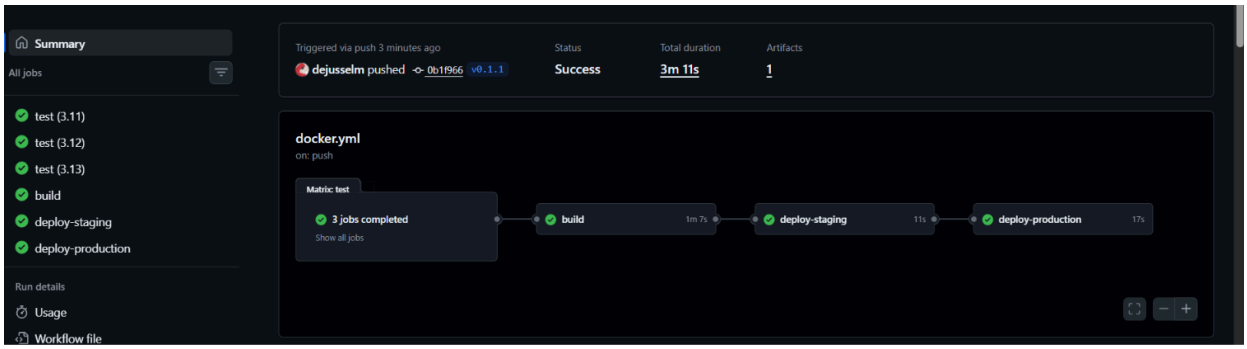


Une image versionnée permet de permettre le bon fonctionnement du code sur toute machine, au lieu que cela marche seulement localement.

TP3-7 Run en attente d'approbation



TP3-7 Run terminé



Un test de fumée permet de s'assurer que l'image a bien démarré
On demande une approbation humaine pour la production comme elle ne doit pas forcément être déployée instantanément

TP3-8 Sortie de commande id

```
PS C:\Users\quasc> docker run --rm tasks-api:local id
uid=1000(default) gid=1000(default) groups=1000(default)
PS C:\Users\quasc>
```

Il faut éviter d'exécuter une application en root dans un conteneur car cela donne des privilèges sur le serveur à n'importe qui, ce qui est très dangereux pour les fichiers

De plus on doit partir du principe du moindre privilège : on donne le minimum de droits nécéssaires

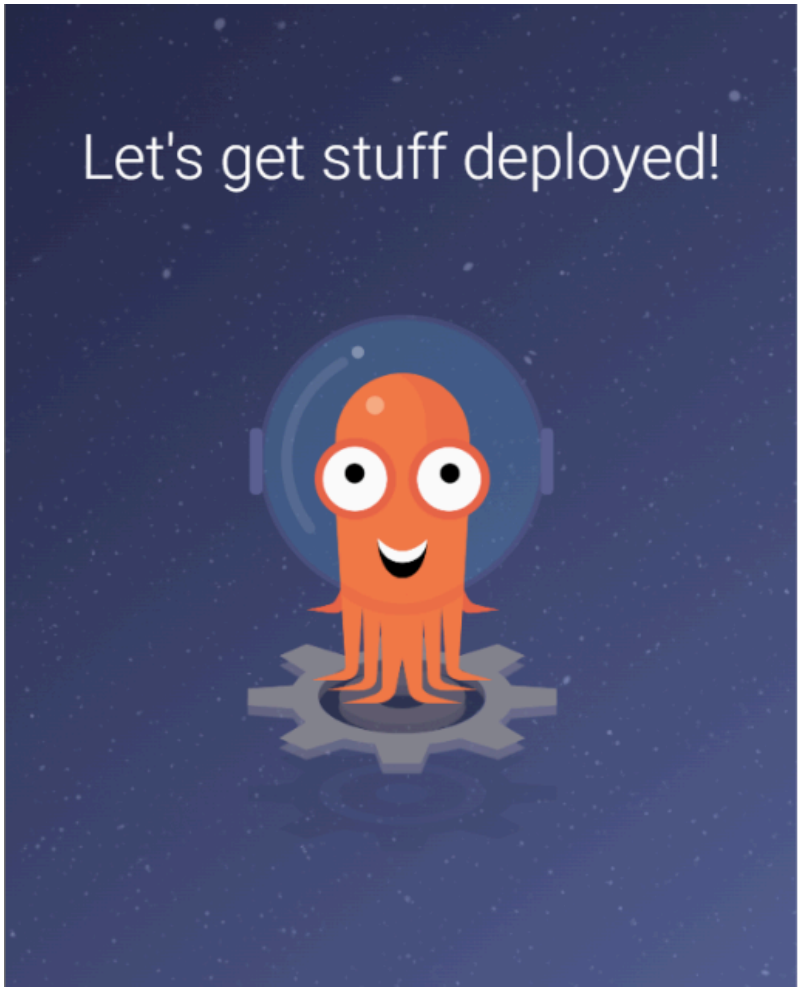
TP4-1 Kubectl get nodes

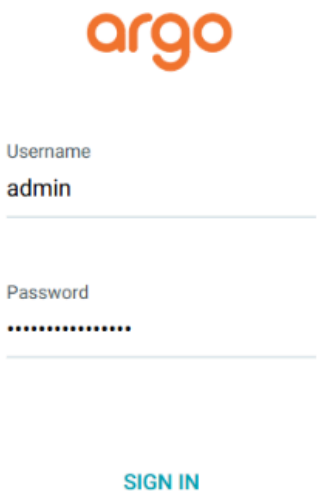
```
PS C:\Users\quasc> kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
gitops-control-plane   Ready    control-plane   14h   v1.37.0
PS C:\Users\quasc>
```

TP4-2 Get pods

```
PS C:\Users\quasc> kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          2m33s
argocd-applicationset-controller-794bb95c9d-mng56   1/1     Running   0          2m33s
argocd-dex-server-66cf64bfcf-vddtv                  1/1     Running   0          2m33s
argocd-notifications-controller-5878c8d644-vp67b    1/1     Running   0          2m33s
argocd-redis-5869b7547-4p2f8                        1/1     Running   0          2m33s
argocd-repo-server-7d55cc466b-nxplv                 1/1     Running   0          2m33s
argocd-server-85b849c688-bhnk8                      1/1     Running   0          2m33s
```

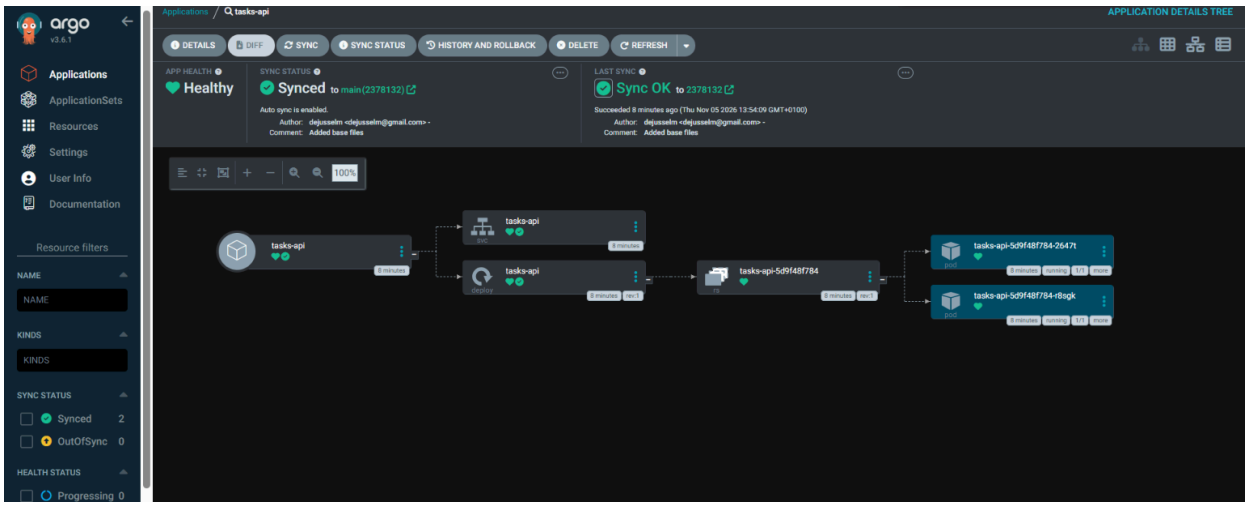
TP4-2 Page de connexion Argo





Un pod est une capsule faisant tourner un ou plusieurs containers, un deployment dirige ces pods
On range les ressources (ex : argocd) dans le namespace

TP4-3 L'application dans Argo CD, état Synced et Healthy, avec ses 2 pods

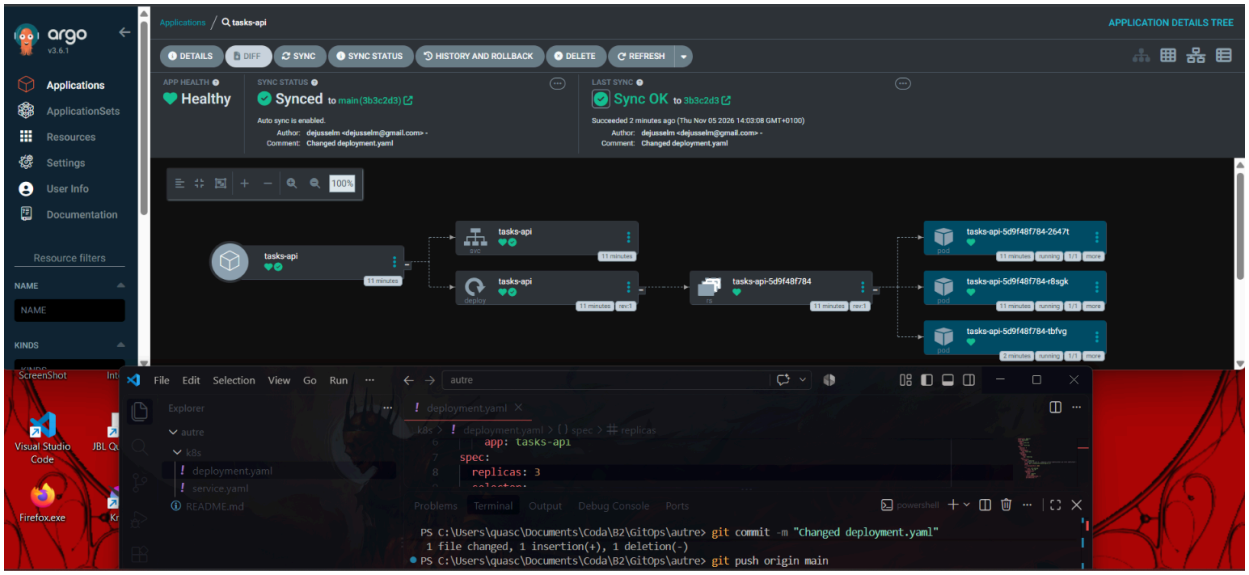


TP4-4 Résultat de l’appel /version

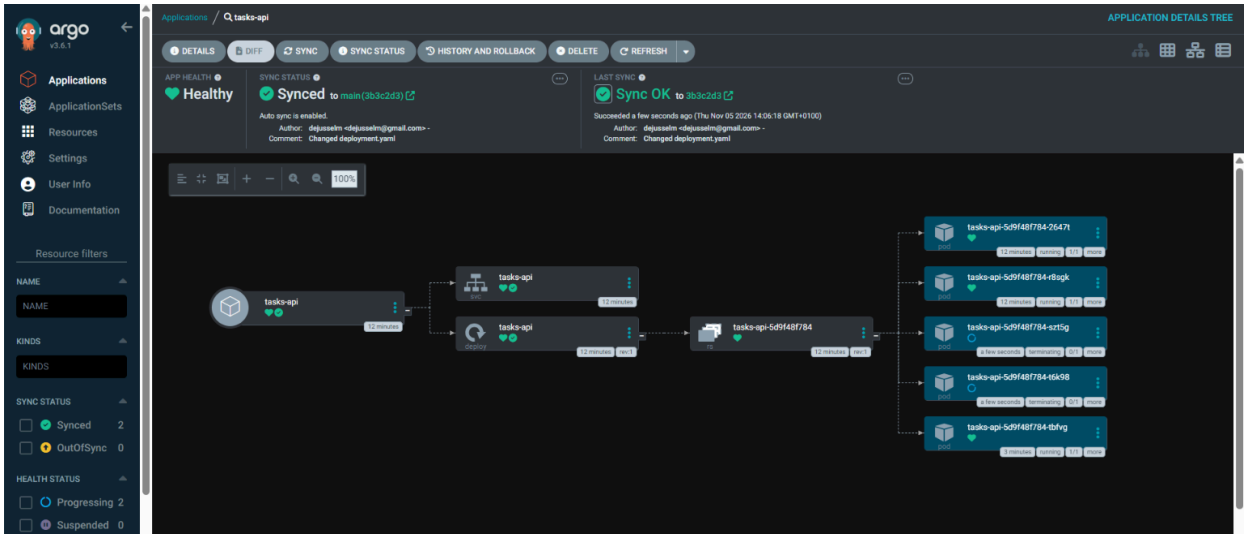
```
PS C:\Users\quasc> curl.exe http://localhost:8000/version
{"version":"v0.1.0"}
PS C:\Users\quasc>
```

On sépare le dépôt de l’application du dépôt de configuration pour éviter de devoir relancer tous les test concernant l’application, à chaque modification

TP4-5 Le commit sur replicas: 3 et les 3 pods dans Argo CD



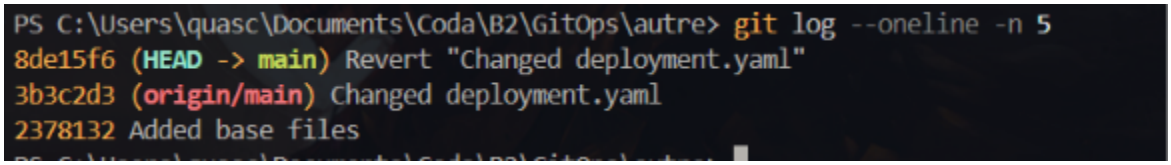
TP4-6 Les pods juste après le kubectl scale à 5



TP4-6 Self Heal



TP4-7 Historique des commit avec le revert

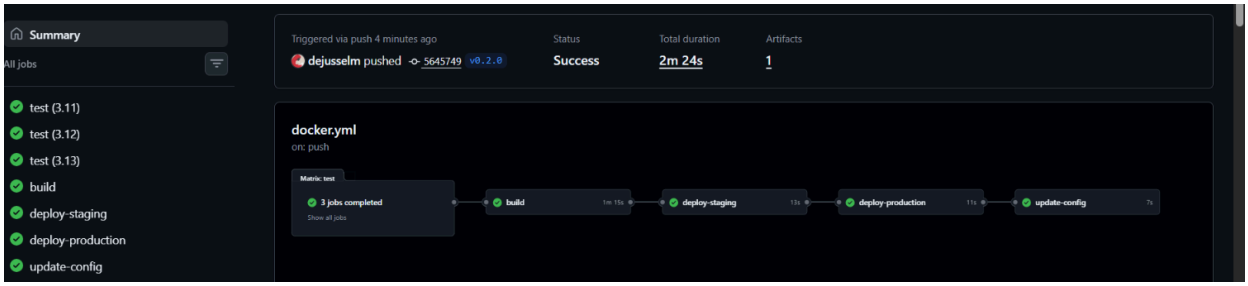


TP4-7 Retour à 2 pods



SelfHeal signifie qu'il se répare tout seul en fonction de ce qui est indiqué dans le dépôt github
Le retour en arrière est grandement simplifié grâce à l'historique qu'est git, et argo cd applique la modification automatiquement sans intervention manuelle

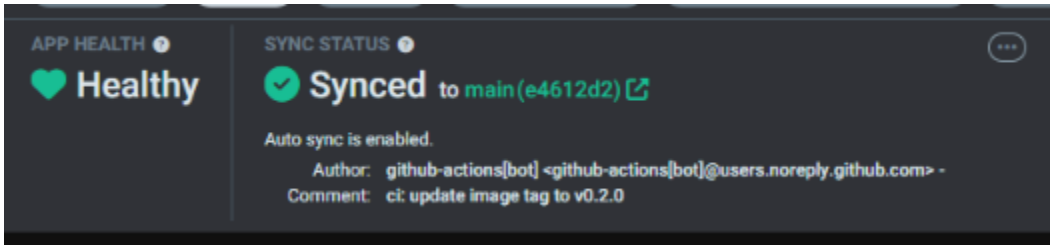
TP4-8 Tous les job au vert



TP4-9 Commit automatique



TP4-9 ArgosCD synchronisé



TP4-10 Version 0.2.0 en appelant /version



Lorsqu'un git tag est push, il déclenche la CI qui se déroule ainsi : elle build l'image docker, la pousse sur le registre ghcr.io, et enfin met à jour le fichier deployment.yaml de tasks-config. Enfin, Argo CD se synchronise automatiquement, et ordonne donc au cluster de déployer la nouvelle version avec les modifications.
C'est la CI qui push le code, et Argo CD qui le pull.
La différence avec un CI / CD classique, c'est qu'Argo CD se base sur l'état du repo, et ne peut donc pas être modifié manuellement : il se réinitialise à la version du repo git (self heal). Il fait donc en sorte que le cluster corresponde toujours au repo, comme il s'agit de la "source de vérité"

Bilan

Ce que j'ai pu retenir :
L'utilité de la page Actions sur GitHub, le principe de pipeline et workflow, la structure d'un fichier .yml dans le dossier .github/workflow, le CI / CD, ArgoCD avec les cluster…

Ce qui m'a posé problème :
Dans le TP3, la partie 3, je ne savais pas comment structurer le fichier docker.yml malgré les consignes : la majorité n'avait pas été abordé en cours
Dans le TP4, la partie 4 notamment avec le script bash : quand je lançais la pipeline, il ne trouvait pas le fichier du script, et j'ai perdu beaucoup trop de temps là dessus. (à voir les 150 workflow échoués pour cette étape)

Ce que je referai autrement :
Essayer de comprendre le fonctionnement des diverses parties du job update-config et envisager des tests en local au lieu de lancer le workflow complet à chaque fois : cela m'aurait fait gagner beaucoup de temps si je l'avais fait dès le début (je l'ai fait trop tard).